

Report on Gene Sequencing

Rakesh Itani
Master's in Computer Science
Idaho State University
Pocatello, ID, 83209, USA
rakeshitani@isu.edu

March 31, 2026

Abstract

This project report summarizes the use of dynamic programming for gene sequencing. The algorithm tries to find the alignment between two sequences.

1 Introduction

This project aims to discuss "Gene sequencing" using the dynamic programming to find the alignments between two sequences. It can be achieved using the Needleman-Wunsch algorithm. It constructs a matrix where cell represents the optimal alignment score of aligning a substring of one sequence with a substring of the other sequences. The algorithm then fills the matrix iteratively considering three operations: match, insert and delete. After fillinf the matrix, the optimal alignment can be tracked back from the bottom-right cell to the top-left cell to obtain the aligned sequences.

2 Pseudocode and Asymptotic Analysis

2.1 Pseudocode for unrestricted alignment

```
1  def unrestricted_alignment(seq1, seq2,
    align_length):
2  # Used to implement Needleman-Wunsch scoring
3  MATCH = -3
4  INDEL = 5
5  SUB = 1
6
7  seq1 = seq1[:align_length]
8  seq2 = seq2[:align_length]
9
10 seq1_len = len(seq1)
11 seq2_len = len(seq2)
12
13 def check_boundaries(row, col):
14     if row < 0 or row > seq1_len or col < 0 or
        col > seq2_len:
15         return False
16     return True
17
18 cost = []
19 # initializing the cost matrix
20 for row in range(seq1_len + 1):
21     current_row = []
22     for col in range(seq2_len + 1):
23         if row == 0 or col == 0:
24             current_row.append(row * INDEL if col
                == 0 else col * INDEL)
25         else:
26             current_row.append(float("inf"))
27     cost.append(current_row)
28
29 backpointer = {}
30 # intializing back pointer
31 for row in range(seq1_len + 1):
```

```

32     for col in range(seq2_len + 1):
33         if col == 0:
34             backpointer[(row, col)] = (row - 1,
35                                         col)
36         elif row == 0:
37             backpointer[(row, col)] = (row, col -
38                                         1)
39     # populating the cost matrix and backpointer
40     for row in range(seq1_len + 1):
41         for col in range(seq2_len + 1):
42             if not check_boundaries(row, col):
43                 continue
44             if check_boundaries(row - 1, col - 1):
45                 cost[row][col] = cost[row - 1][col -
46                                         1] + (
47                     MATCH if seq1[row - 1] ==
48                         seq2[col - 1] else SUB
49                 )
50             backpointer[(row, col)] = (row - 1,
51                                         col - 1)
52         if check_boundaries(row - 1, col):
53             prev_above_cost = cost[row - 1][col]
54             new_above_cost = prev_above_cost +
55                 INDEL
56             if new_above_cost <= cost[row][col]:
57                 cost[row][col] = new_above_cost
58                 backpointer[(row, col)] = (row -
59                                         1, col)
60         if check_boundaries(row, col - 1):
61             prev_left_cost = cost[row][col - 1]
62             new_left_cost = prev_left_cost + INDEL
63             if new_left_cost <= cost[row][col]:
64                 cost[row][col] = new_left_cost

```

```

62             backpointer[(row, col)] = (row,
63                                     col - 1)
64
65 # aligning sequences
66 alignment1 = ""
67 alignment2 = ""
68 (cur_row, cur_col) = (seq1_len, seq2_len)
69 while cur_row > 0 or cur_col > 0:
70     (prev_row, prev_col) = backpointer[(cur_row,
71                                         cur_col)]
72     if prev_row < cur_row and prev_col < cur_col:
73         alignment1 += seq1[cur_row - 1]
74         alignment2 += seq2[cur_col - 1]
75     elif prev_row < cur_row:
76         alignment1 += seq1[cur_row - 1]
77         alignment2 += "-"
78     elif prev_col < cur_col:
79         alignment1 += "-"
80         alignment2 += seq2[cur_col - 1]
81     (cur_row, cur_col) = (prev_row, prev_col)
82
83 alignment1 = alignment1[::-1][:100]
84 alignment2 = alignment2[::-1][:100]
85
86 score = cost[-1][-1]
87 return score, alignment1, alignment2

```

Time complexity and space complexity

Using unrestricted alignment

1. initializing the cost and backpointer takes $O(n*m)$ time and $O(n*m)$ space
2. populating the cost matrix and backpointer takes $O(n*m)$ time and $O(n*m)$ space
3. aligning the sequences takes $O(n)$ times and $O(n)$ space

$$\begin{aligned} \text{Time_complexity} &= O(n * m) + O(n * m) + O(n) \approx O(n * m) \\ \text{Space_complexity} &= O(n * m) + O(n * m) + O(n) \approx O(n * m) \end{aligned} \quad (1)$$

2.2 Pseudocode for banded alignment

```
1  def banded_alignment(seq1, seq2, align_length,
    banded):
2  # Used to compute the bandwidth for banded version
3  MAXINDELS = 3
4
5  # Used to implement Needleman-Wunsch scoring
6  MATCH = -3
7  INDEL = 5
8  SUB = 1
9
10 seq1 = seq1[:align_length]
11 seq2 = seq2[:align_length]
12
13 seq1_len = len(seq1)
14 seq2_len = len(seq2)
15
16 def check_boundaries(row, col):
17     if row < 0 or row > seq1_len or col < 0 or
        col > seq2_len:
18         return False
19     if banded == True:
20         # check if row and col is within the
            banded region
21         if col < row - MAXINDELS or col > row +
            MAXINDELS:
22             return False
23         if row < col - MAXINDELS or row > col +
            MAXINDELS:
24             return False
25     return True
26
27 # adjusting the column
28 def adjust_column(row, col):
29     if row > MAXINDELS:
30         return col - (row - MAXINDELS)
31     return col
```

```

32
33     # check sequence lengths and return inf if true
34     if abs(seq1_len - seq2_len) > MAXINDELS:
35         return float("inf"), "No Alignmnet Possible",
            "No Alignment Possible"
36
37     cost = []
38     # initializing cost matrix
39     for row in range(seq1_len + 1):
40         row_values = []
41         for col in range(row - MAXINDELS, row +
            MAXINDELS + 1):
42             if check_boundaries(row, col):
43                 if col == 0:
44                     value = row * INDEL
45                 elif row == 0:
46                     value = col * INDEL
47                 else:
48                     value = float("inf")
49                 row_values.append(value)
50         cost.append(row_values)
51
52     backpointer = {}
53     # intializing back pointer
54     for row in range(seq1_len + 1):
55         for col in range(seq2_len + 1):
56             if col == 0:
57                 backpointer[(row, col)] = (row - 1,
                    col)
58             elif row == 0:
59                 backpointer[(row, col)] = (row, col -
                    1)
60
61     # populating the cost matrix and backpointer
62     for row in range(seq1_len + 1):
63         for col in range(row - MAXINDELS, row +
            MAXINDELS + 1):
64             if not check_boundaries(row, col):

```

```

65         continue
66
67         (row_curr, col_curr) = (row,
68                                 adjust_column(row, col))
69
70         if check_boundaries(row - 1, col - 1):
71             cost[row_curr][col_curr] = cost[row -
72             1][
73                 adjust_column(row - 1, col - 1)
74             ] + (MATCH if seq1[row - 1] ==
75                 seq2[col - 1] else SUB)
76             backpointer[(row, col)] = (row - 1,
77                                         col - 1)
78
79         if check_boundaries(row - 1, col):
80             prev_above_cost = cost[row -
81             1][adjust_column(row - 1, col)]
82             new_above_cost = prev_above_cost +
83                 INDEL
84             if new_above_cost <=
85                 cost[row_curr][col_curr]:
86                 cost[row_curr][col_curr] =
87                     new_above_cost
88                 backpointer[(row, col)] = (row -
89                                         1, col)
90
91         if check_boundaries(row, col - 1):
92             prev_left_cost =
93                 cost[row][adjust_column(row, col -
94                 1)]
95             new_left_cost = prev_left_cost + INDEL
96             if new_left_cost <=
97                 cost[row_curr][col_curr]:
98                 cost[row_curr][col_curr] =
99                     new_left_cost
100                 backpointer[(row, col)] = (row,
101                                         col - 1)

```

```

89     alignment1 = ""
90     alignment2 = ""
91     (cur_row, cur_col) = (seq1_len, seq2_len)
92     while cur_row > 0 or cur_col > 0:
93         (prev_row, prev_col) = backpointer[(cur_row,
94                                             cur_col)]
95         if prev_row < cur_row and prev_col < cur_col:
96             alignment1 += seq1[cur_row - 1]
97             alignment2 += seq2[cur_col - 1]
98         elif prev_row < cur_row:
99             alignment1 += seq1[cur_row - 1]
100            alignment2 += "-"
101        elif prev_col < cur_col:
102            alignment1 += "-"
103            alignment2 += seq2[cur_col - 1]
104        (cur_row, cur_col) = (prev_row, prev_col)
105
106    alignment1 = alignment1[::-1][:100]
107    alignment2 = alignment2[::-1][:100]
108
109    score = cost[-1][-1]
110    return score, alignment1, alignment2

```

Time complexity and space complexity

Using unrestricted alignment

1. initializing the cost and backpointer takes $O(n * k)$ time and $O(n * k)$ space
2. populating the cost matrix and backpointer takes $O(n * k)$ time and $O(n * k)$ space
3. aligning the sequences takes $O(n)$ times and $O(n)$ space

$$\begin{aligned}
 \text{Time_complexity} &= O(n * k) + O(n * k) + O(n) \approx O(n * k) \\
 \text{Space_complexity} &= O(n * k) + O(n * k) + O(n) \approx O(n * k)
 \end{aligned} \tag{2}$$

3 Observations and Results

3.1 Observation

In this project, both the unrestricted and banded algorithm was used to align the sequences. In order to populate the cost matrix, first of all boundaries were checked i.e. checked if the current matrices of cost matrix is within the valid region (in case of banded, the valid region is computed using $2 * MAXINDELS + 1$). Then, the cost for the diagonal was computed, above and left (i.e. MATCH or SUB, INDEL operation) and updated the cost matrix at current indices by the lowest value among the three.

After populating the cost matrix, two empty string variables were initialized to hold the alignments. Using the backpointer, the current version of either diagonal was compared and the alignment was computed accordingly.

If the diagonal matched the current value, $sequence1[current - 1]$ was appended to alignment 1 and $sequence2[current]$ was appended to alignment 2.

If the above matches the current value, $sequence1[current - 1]$ was appended to alignment 1 and a gap in sequence 2.

If the left matches the current value $sequence2[current - 1]$ was appended to alignment2 and a gap in sequence 1.

First 100 characters of sequence3 and sequence10 computed using the unrestricted alignment with k = 1000

sequence3: gattgcgagcgattgcgtgcgtgcatcccgcttc-actg-at-ctcttgtagatctttcataatctaaactttataaaaa
catccactccctgta-

sequence10:-ataa-gagtgattggcggtccgtacgtaccctttctactctcaaactcttgtagtttaaadc-taatctaaactttataaa-
cggc-acttctgtgt

First 100 characters of sequence3 and sequence10 computed using the banded alignment with k = 3000

sequence3: gattgcgagcgattgcgtgcgtgcatcccgcttc-actg-at-ctcttgtagatctttcataatctaaactttataaaaa
catccactccctgta-

sequence10:-ataa-gagtgattggcggtccgtacgtaccctttctactctcaaactcttgtagtttaaadc-taatctaaactttataaa-
cggc-acttctgtgt

3.2 Results

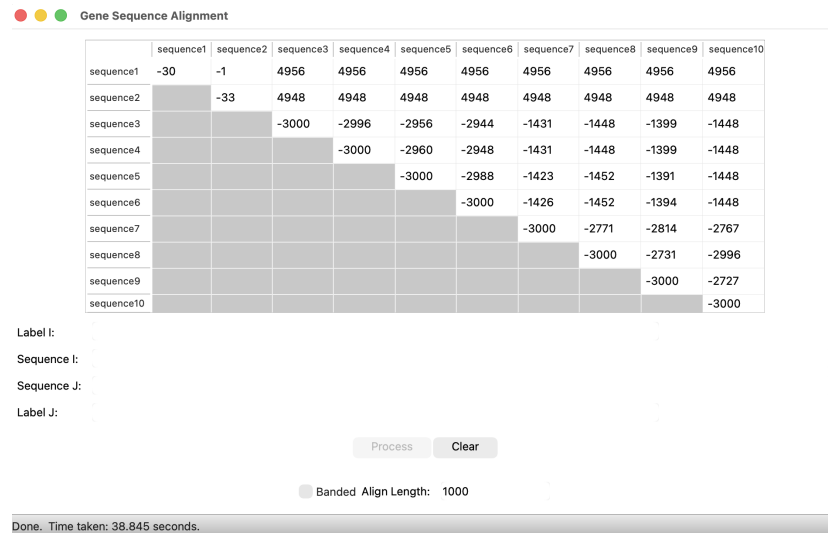


Figure 1: Unrestricted alignment for length 1000

The figure (in figure 1) shows the optimal cost for each sequences and the time taken to solve the problem. Here, the align length is 1000 and it took 38.845 seconds to solve it using unrestricted alignment.

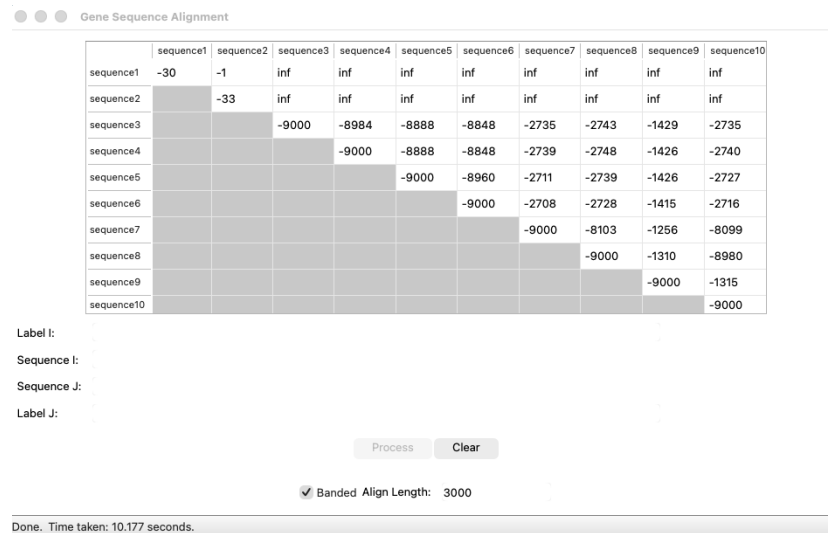


Figure 2: Banded alignment for align length 3000

The figure (in figure 2) shows the optimal cost for each sequences and the time taken to solve the problem. Here, the align length is 3000 and it took 10.177 seconds to solve it using banded alignment.

4 Conclusion

The successful completion of this project ensured the generation of code to compute both the unrestricted and banded alignment algorithm. The time taken by the program to compute and solve the given sequences were successfully calculated and the banded calculations were found to be much more efficient compared to the unrestricted ones.